Raag Studio

Product requirements and implementation specification

Working name. Version 0.1. 24 October 2026. Owner: Siddhant Pathak. **Status:** Proposed product; no application built or deployed by this task.

The product in one sentence

A music studio for people who can describe what they want to feel, but do not know how to compose: describe an idea, get an understandable musical plan, generate music, and refine the parts that matter without losing a take you love.

1. Product decisions

- The requirements text box is the only universally required field. Everything else has an Auto state. Providing exact lyrics or an upload becomes required only when the user chooses that workflow.
- One project has a rich, editable musical specification. Each music engine receives only the instructions it can use. More stored detail must not automatically mean a longer prompt.
- Every control distinguishes a creative request from an actual audio edit. A slider labelled "flute volume" is an audio mixer only when we have a usable flute track; otherwise it is a request for a new performance.
- Beginner controls describe experiences. Producer controls reveal musical terms, examples and limits. They are views of the same plan, not competing settings.
- Retain the accepted recording. Changing the brief, trying another model or repairing a section creates a version, not an overwrite.
- Local generation is the first implementation route. Cloud adapters are optional future integrations, with explicit account access, costs and capability checks.

The advantage is an approachable producer workflow, an Indian-instrument vocabulary and continuity with music the listener has accepted. This research does not establish that the feature combination is unique. Several existing products already offer conversational generation and detailed editing. [S1-S7]

2. Evidence from our own music

Result	Listener response	Product consequence
Original Hindi romantic song	Enjoyed; some pronunciation unclear	Treat melody, emotion and diction as separate review dimensions
Spoken-sounding replacement over Sonic Pi	Rejected despite clearer voice/backing	Do not substitute speech synthesis for a singing model
Daylight	"Very enjoyable"	Save as an accepted style example, not a universal quality benchmark
Yamuna at Dawn, refined flute take	Strongly liked for its melody	Save its exact recording and recipe as an accepted anchor
Early Yamuna take	Acoustic checks gave inconsistent flute indications	Reducing competing layers is a useful candidate strategy, not a proven general rule

We can preserve these recipes immediately. Using an accepted recording as an audio reference is a separate operation, depending on the selected model. A recipe preset is not an audio-preserving continuation. Earshot descriptions and level checks support review; they do not decide taste or certify virtuosity.

3. Audience, goals and exclusions

Primary user: a beginner who says "a beautiful flute melody, gentle rhythm, no singing" or "a romantic Hindi song using these exact words."

Secondary users: video creators who need a cue of a specified length; enthusiasts who want to explore orchestration; producers who need a draft they can export and finish elsewhere.

Goals: produce an enjoyable first result without music theory; make the plan understandable; expose progressively deeper control; preserve good performances; make lyrics policy explicit; make cost, time and engine limitations visible.

Not MVP goals: a full DAW, live collaboration, distribution to streaming services, custom-model training, exact orchestra simulation, automatic copyright clearance, a guaranteed singer identity, professional mastery from a checkbox, or a claim that every language sings equally well.

The first release is a personal local web app on Siddhant's Mac. A service supporting multiple remote users needs authentication, tenancy, quotas and a hosted generation fleet; that is a later product scope.

4. Main experience

Create

A large box asks: **"Describe the music you want. Tell us the feeling, instruments, story, or anything you want to avoid."** Example: "A soulful bansuri by a river at dawn. Tabla should join gently. Give the flute expressive bends and quick runs, but keep it peaceful. No singing. About three minutes."

Below it, show optional chips: **Sound**, **Instruments**, **Voices**, **Lyrics**, **Length**. Defaults are Auto. Never require genre, BPM, key or music vocabulary.

Create music runs interpretation, validation and generation in one action. The resulting plan is visible and editable, but is not a mandatory approval step. A separate **Plan only** action lets users inspect it without rendering. Lyrics intentionally authored by the user are never rewritten as part of interpretation.

Understand the plan

Show a short summary: "Warm Indian acoustic piece. Bansuri leads; tabla and santoor support. Instrumental, with a quieter middle and an expressive ending." Each setting has one provenance label: **You chose**, **AI suggested**, or **Preset**. Expand to see the reason; do not fill the screen with implementation data.

Listen and refine

Display the audio player, versions and actions: **More like this**, **Change the arrangement**, **Improve a section**, **Make another take**, **Save favourite**, **Download**. Suggested changes are specific: "less busy accompaniment," "bring the choir in later," or "keep this melody, explore another arrangement."

Before a revision starts, describe its actual scope: whole new performance, section replacement, or mix-only change. "Keep this melody" is a goal unless a supported score/reference workflow enforces it. The original recording always remains playable.

5. Progressive disclosure

View	What users see	Default behavior
Simple	Required brief, five optional chips, plan summary and player	AI fills unspecified choices
Shape the music	Instrument roles, voice/lyrics choices, mood/energy, broad section cards	Only relevant controls appear
Producer view	Technique, register, groove, harmony, relationships, effects and timeline intent	Advanced settings still allow Auto
Engine details	Model version, seed, supported operations and render settings	Developer/troubleshooting drawer

Every musical term gets a plain-language explanation. Where helpful, add a short owned/licensed example. "Smooth and connected (legato)" is more useful than an unexplained articulation menu. Examples must not silently launch an expensive generation.

6. Instrument selection and performance

The catalogue supports families, search, cultural names/aliases and a small audition example. Initial catalogue: piano/keys; acoustic/electric guitars; bass; violin/viola/cello and strings; flute/bansuri; sitar/santoor/sarod/veena; harmonium/tanpura; trumpet/horn/trombone/brass; tabla/ghatam/mridangam/pakhawaj; drum kit/taiko/shakers; harp/mallets; pads/plucks/synth bass/noise textures. Listing an instrument does not establish reliable rendering by every engine.

Each selected instrument has:

- **Role:** main melody, answering melody, harmony, rhythmic pattern, bass, drone, texture, accent or solo passage.
- **Priority:** essential, preferred or optional; independent of loudness. Essential means generation must be reviewed for that requirement, not that presence is guaranteed.
- **Presence:** subtle, balanced or featured. This is prompt guidance until an editable track exists.
- **Character:** warm/bright/dark/airy/round/rough, offered only when relevant.
- **Performance:** simple, lyrical, expressive, ornate or agile; plus sparse/balanced/busy phrasing. "Virtuosic" is a performance direction, not a measured skill rating.
- **Register and articulation:** Auto/low/middle/high; connected, short, plucked, bowed, sustained, tremolo, etc., appropriate to the instrument.
- **Entrances/exits:** whole piece or named sections; later phases allow exact timeline placement.

Instrument-specific detail examples: bansuri bends, grace notes and breath-shaped phrases; guitar fingerpicking/strumming; strings legato/pizzicato; brass sustained/short accented phrases; synth pad/pluck/growl/filter movement. A specific raga or tala remains experimental unless the selected route is evaluated for it.

The app can accept many instruments, then recommend a hierarchy: "Bansuri leads; santoor answers; tabla supplies rhythm; strings enter only at the high point." Warn about crowding in plain language rather than enforcing an arbitrary three-instrument cap. An optional selected instrument must never displace an explicitly essential one during prompt compression.

7. Voices, choir and the word "chorus"

Use separate labels for **song refrain** and **background voices / choir**. A repeated chorus section does not require a choir; a choir can appear in an instrumental-style cue without lexical lyrics.

Control	Options and interpretation
Voice mode	Auto, instruments only, wordless voices, sung lyrics, rap/spoken delivery
Lead	None, solo, duet or small ensemble; generic tone/register preferences
Background voices	Off, light harmonies, small choir, large choir or Auto
Scale	Solo 1, duet 2, small group about 3-8, chamber choir about 8-24, large choir about 25-60, massed choir about 60-100+
Exact requested count	Optional target, e.g. 100, explicitly approximate in generative mode
Vocal content	Ooh/aah/hum; sing specified words; chant; sustained vowels; Auto
Relationship	Support with held harmony, follow the lead melody, answer it, sing an independent counterline, or alternate sections
Voice distribution	Auto, low voices, high voices, mixed; SATB only in Producer view
Placement	Subtle background, blended or featured; arrive in specific sections

The choir planner considers key, chord movement, melodic space, relative register, rhythm and section density. It can choose held harmony under an ornate flute lead, or answering phrases between the flute's breaths. This is a compositional intention; exact note matching requires a score-conditioned workflow. We must not pretend prompt text guarantees independent choir voices, precise SATB parts, a counted ensemble or unchanged instrument stems.

An "instruments only" choice disables all voices. Wordless choir switches the engine's instrumental boolean off where required, while keeping lexical lyrics absent. Changes cannot leave a hidden instrumental flag suppressing a requested choir.

8. Lyrics workflows and language policy

Mode	User provides	App may do
No lyrics	Nothing	Render instruments or wordless voice according to voice mode
Write for me	Topic, story, language and optional style preferences	Draft original lyric text, structure it and render when requested
Use my words exactly	Full lyrics	Keep lexical content and order; no substitutions, translations or extra repeated lines without permission
Adapt my lyrics	Draft plus allowed edits	Improve singability/rhyme while showing a diff and preserving locked lines
Inspired, never copied	Themes, mood or a reference	Derive an abstract writing brief, then write a new lyric; no copy-and-replace paraphrase of a famous song
Mixed language	Language by section/line	Preserve code-switching; review pronunciation separately for each language

Exact mode stores an immutable text source and a hash. Structural tags live separately. A proposed repeat, altered syllable, new interjection or translation must be explicit; the engine's performed words can still differ. Long exact lyrics trigger a duration-fit check: suggest a longer song or more sections, never silently delete words. A word that seems wrong in an ASR transcript is flagged for listening, not automatically declared a singing error.

Useful writing controls: theme, emotional perspective, story arc, imagery, simplicity/poetry, rhyme density, syllabic flow, refrain, repeated hook, language/script, formality, explicitness, forbidden words, line locks and section locks. Store pronunciation hints separately from the display lyric; do not silently replace the user's authoritative spelling.

Launch commitment: original lyric drafting in Hindi and English, with editorial review. Hindi text may use Devanagari or Roman transliteration. Urdu vocabulary and Hinglish can be offered with clear review needs. Punjabi and other languages start as experimental rather than launch-quality promises. The assistant can draft in additional languages, including Spanish, French and Portuguese, but poetic idiom and prosody need native-speaker review before we promote them.

Maintain three distinct statuses per language: **text drafting**, **engine accepts input**, and **singing/diction evaluated**. Our Hindi song was liked but had pronunciation issues; English wordless vocals do not validate English lyrical singing. Do not turn a model's multilingual list into a confidence percentage or claim production-quality singing in dozens of languages.

9. Musical direction without overwhelming the user

Simple control	Optional producer detail	Current interpretation
Feeling	Mood, tension, emotional arc	High-value prompt guidance
Energy	Dynamics and density by section	Broad guidance; real automation requires tracks
Pace	BPM, groove, note activity, half-time/double-time feel	Separate dimensions; a half-time drum feel need not change BPM
Catchy vs exploratory	Recurring motif, variation, improvisation	Prompt guidance; exact melody needs notes/reference
Calm vs dramatic	Contrast, climax, drop and recovery	Prefer a few defining events
Open vs full	Texture, orchestration and register spacing	Avoid continuous doubling of every instrument
Familiar vs adventurous	Harmonic colour, modal mixture, modulations	Experimental unless score-controlled
Intimate vs vast	Dry/room/hall reverb, stereo width	Sound direction initially; real effects later
Acoustic vs electronic	Instrument balance and sound-design palette	Genre fusion with explicit lead/support roles
Ending	Resolve, gently thin out, sudden stop, loop, unresolved	Loop seams and duration verified in finishing

More technical options belong in Producer view: meter, swing, syncopation, ostinato, drone/pedal note, scale/mode, chord progression, melodic range, countermelody, call-and-response, chromatic tension, articulation, octave changes, pickup, cadence and silence/breath. Do not interpret "expert musician" as a command to maximize speed, notes or density.

The default planner selects a coherent subset. A cinematic request might use brass register contrast, restrained taiko, a brief quiet section and a choir arrival. It does not need every orchestration technique in the vocabulary.

10. What Joey Nato's breakdowns add

Research uses the user's Ashes transcript plus two additional caption transcripts obtained through Agent Reach. Auto-captions misrecognize words; observations below are paraphrased, not a verified transcription of the score. The Raiden subtitle attempt returned a rate-limit response and was not retried; further YouTube requests stopped. [J1-J3]

Observation	Friendly app choice	Where it belongs
Ashes: brass shifts into a lower register	"Make the brass darker and heavier"	Instrument register/tone; soft guidance
Ashes: reverse synth, pluck and noise colours	"Add a rising/reversed texture"	Sound design; exact reversal only on isolated audio
Ashes: drums and cymbals grow more intense	"Build percussion gradually"	Section arc; measurable automation only with tracks
Ashes: a piano idea returns with more motion	"Bring the theme back with variation"	Motif development; not necessarily a tempo change

Observation	Friendly app choice	Where it belongs
Ashes: low connected strings support upper lines	"Warm support under the melody"	Role/register/articulation
Ashes: choir and strings follow the same line	"Let choir follow this melody"	Relationship; exact doubling needs a score
Ashes: electronic elements combine with orchestra	"Orchestral + electronic"	Palette/genre blend
Ashes: quiet passage followed by a large release	"Pull back, then bloom"	Section contrast
YouSeeBIGGIRL: changing layer prominence and recurring material	"Choose what leads; return to the theme"	Orchestration and development
YouSeeBIGGIRL: sudden accented arrival, bending synth, lead/choir contrast	"Surprise arrival; contrasting voice roles"	Section transition and vocal relationship
La Signora: left/right parts, plucked strings and moving harp figures	"Separate instruments in space"	Advanced mix; exact panning needs separate parts
La Signora: escalating layers/register, snare rolls and changing rhythmic feel	"Build momentum, then switch the groove"	Energy and rhythmic feel

Some names are hypotheses, not authoritative instrument identifications: do not map every cinematic impact to taiko, or label every bell-like timbre a glockenspiel. The best product lesson is **relationships over inventory**: what enters, what answers, what supports, what changes, and when the arrangement leaves space.

11. Capability truth: can AI follow all this?

Yes to storing a rich musical plan. No to promising that a single audio model will obey every detail simultaneously.

Control class	Examples	Product behavior
Actual file operation	Export format, exact trim, whole-mix gain/fade	Deterministic and verifiable
Engine input	Seed, duration target, language code, checkpoint	Passed exactly; musical output still needs review
Soft musical direction	Bansuri, large choir, legato, romantic mood, ornate playing	Label as creative guidance; never certify solely from the prompt
Needs another route	Exact notes, independent part levels, perfectly aligned unison	Enable only with score/tracks or mark unavailable
Currently unverified	Strict raga grammar, exact 100-person choir, identical voice during repair	Experimental target; no success guarantee

The installed ACE turbo route documents short captions and structured metadata. Our working wrappers cap captions at 512 characters; this is a current adapter budget, not a universal rule for all future models. Whole-song generation does not produce individually editable instrument tracks by default. A stem separator produces estimated parts, not guaranteed original isolated recordings. [S8-S9]

Important limitations: do not expose turbo guidance scale as "follow my instructions harder" when that checkpoint overrides it; do not advertise base-only extraction/layer tasks as available from the installed turbo weights; do not use VoiceStudio speech as a singing fallback. Score-conditioned models and a paid section-plan API are possible future routes, not validated parts of today's installed application. [S8-S11]

12. Brief-to-plan compiler

- Preserve the brief, exact lyrics and manual choices.
- Extract intent into a versioned structured plan; fill absent fields with suitable Auto decisions.
- Record provenance, priority and lock status for each explicit choice. Engine support and interpretation confidence are separate fields.
- Resolve ordinary tradeoffs automatically. If a new brief directly contradicts a locked choice, show a small inline conflict, rather than silently changing either.
- Allocate instrument roles, register space, voice relationships and a small section arc. Treat silence and removing layers as valid decisions.
- Classify requested controls against the selected engine's capability registry.
- Compile a compact engine request: lead identity, mood/style, essential support, rhythm and two or three defining developments. Keep the remaining plan for review/future edits. The "few key instructions" policy is a proposed heuristic that needs evaluation, not a proven universal optimum.
- Show any essential instructions that cannot fit or cannot be executed. Never silently truncate exact lyrics or drop essential instruments.
- Render the agreed number of takes within the job budget. Technical failures may be retried once under an explicit policy; creative dissatisfaction never starts an unlimited loop.
- Check files and offer a comparison. Listener acceptance selects the winner.

Precedence: direct manual lock > latest explicit user edit > accepted project preference > preset > inferred default. A lock protects configuration; it does not magically lock the waveform. Imported audio is immutable until an edit creates a derivative.

Examples of conflicts: "no voices" plus enabled choir; solo flute plus many equally featured leads; exact long lyrics plus a 20-second duration; exact original melody plus an unconditioned whole-song generation. The app offers a coherent resolution with the smallest necessary change.

13. Generation and editing architecture

Browser UI -> local application API -> planner/compiler -> capability adapter -> durable job queue -> local music worker -> audio finishing/checks -> version store/player.

Proposed stack: React/TypeScript interface, Python/FastAPI orchestration, SQLite job/project metadata, filesystem asset store, one native ACE worker on the Mac, FFmpeg for finishing. The planner uses an explicitly configured LLM API or local model; access through this chat does not automatically provide an app with API credentials or subscription-inclusive inference.

Use one heavy local generation job at a time initially. Verify free resources and any shared video queue before loading models. Cancellation is best-effort during a non-interruptible inference stage; never promise instant GPU release. Save outputs and job state atomically, and recover jobs after a restart without duplicating paid submissions.

Persist projects, revisions, plans, field provenance, lyric sources/hashes, asset hashes, generation requests, model/runtime revisions, timings, checks and human preferences. Same seed can help repeatability but is not guaranteed across runtime/model changes.

Minimal API contract: `POST /projects`, `POST /projects/{id}/plans`, `POST /projects/{id}/jobs`, `GET /jobs/{id}`, `POST /jobs/{id}/cancel`, `GET /projects/{id}/versions`, `POST /versions/{id}/revisions`, and `GET /assets/{id}`. New generation jobs carry an idempotency key and plan revision; parameter changes after queuing create a new job.

Section repair is a pilot feature. Preserve the original outside an edit by assembling a verified replacement span locally, including documented crossfade handles. Assert sample equality outside the declared changed region. Do not advertise preserved melody or singer inside the generated region. Exact instrument rebalancing remains unavailable without a usable stem. [S11]

For a future hosted service: separate each user's jobs, credentials, assets and storage paths; authenticate downloads; validate uploads; redact secrets from logs; apply deletion/retention settings. No automatic public publishing or third-party sharing.

14. Core functional requirements

ID	Priority	Requirement
F01	P0	Nonempty required brief; Auto everywhere else; Plan only and Create music
F02	P0	Editable, versioned plan with provenance, locks and explainable defaults
F03	P0	Instrument search, roles, priorities and relative presence requests
F04	P0	Instruments/wordless/lyrics modes; choir presets distinct from song refrain
F05	P0	Hindi/English drafting and exact/adapt/inspired lyric policies with source preservation
F06	P0	Compact validated ACE request, no misleading unavailable controls
F07	P0	Queue/progress/cancel, resource checks, restart-safe requests
F08	P0	Player, favourites, immutable versions and optional loudness-matched comparison
F09	P0	MP3/WAV export, technical checks and portable project recipe
F10	P0	Text revision with declared scope: new take versus actual audio operation
F11	P1	Bounded section repair pilot, seam review and preserved-outside-region checks
F12	P1	Section cards, richer technique catalogue, captions and draft lyric timing
F13	P1	Estimated stem extraction and real mix controls for available tracks
F14	P2	Cloud engines, score/MIDI control, humming input and video-scoring workflows

Implementation backlog should split each feature into UI, plan semantics, adapter behavior, persistence and acceptance checks. An attractive control without an execution path is not a completed feature.

15. Quality, cost and acceptance

Maintain separate panels: **Technical checks** (decode, silence, finite samples, clipping, true peak, duration) and **Listener feedback** (melody, emotion, requested lead, rhythm, voice, diction, development). Do not collapse them into a "professional score."

Optional transcription checks flag omissions/repetitions against locked lyrics. Keep raw transcripts unchanged; do not correct ASR against the intended lyric and then call it a successful pronunciation test. Instrument tags are hints, not proof; human review decides whether bansuri is recognizable or beautiful.

Local jobs show measured runtime ranges after enough jobs exist, with model load versus inference time separated. Do not reuse video latency estimates or promise a universal seconds-per-song rate. Show planner, cloud, storage and local-compute costs separately; do not assume a ChatGPT or Claude subscription includes app API usage. Estimate costs before a paid operation and obey the user's configured ceiling.

Proposed launch targets, to validate rather than present as achieved: most first-time testers complete creation without help; a majority find one enjoyable result within two takes; zero silent edits to exact lyrics; zero favourite overwrites; all delivered files decode; every advanced control has a capability status. Measure median/P90 job time and rejected-request rate. Run blind level-matched comparisons when assessing model/prompt changes.

Acceptance scenarios

- Brief alone generates a complete editable plan and a playable result without a music-theory question.
- Bansuri + tabla + no singing compiles to a flute-priority instrumental request; any suspected voice content is flagged, not hidden.
- Selecting a massed choir produces a large-ensemble direction and an approximation note, never a fabricated 100-part count.
- Exact Hindi lyrics survive planning and prompt compilation byte-for-byte; an overlong lyric creates an explicit duration-fit issue.
- "Make it half-time" changes the rhythmic-feel field rather than silently halving BPM.
- "Keep this take, change only the last 12 seconds" uses a supported repair path or clearly offers a whole new take; the original remains intact.
- A flute-level fader cannot claim independent control when only a stereo master exists.
- Changing the brief preserves manual locks and exposes direct conflicts.
- Duplicate Create submissions with the same idempotency key do not start two jobs.
- A rejected or failed generation does not enter an unbounded retry loop or consume an unapproved paid backend.

16. Exciting options worth building later

Idea	User experience	Required foundation
Musical conversation	"Let flute ask and violin answer"	Relationship planner; score/section audition
Emotion journey	Draw calm -> yearning -> hope	Section-aware generation and review
Keep the magic	Protect a favourite phrase while changing another part	Immutable reference, region editing, seam checks
Humming to ensemble	Hum a tune and arrange it	Transcription, melody conditioning and timing checks
Instrument audition carousel	Hear several arrangements of one melody	Score/reference-conditioned rendering; new text takes do not suffice
Voice choir designer	Intimate harmony -> massed choir	Approximate generative texture first; exact parts require synthesis/tracks
Hear the term	Compare legato/staccato, solo/ensemble, dry/hall	Short owned examples with clear labels
Cinematic cue points	"The reveal is at 18 seconds"	Timeline generation or explicit audio editing
Theme and variations	Dawn, rain and celebration versions of a motif	Persistent score/reference with measured adherence
Controlled surprise	Add one unusual but compatible element	Priority-preserving novelty budget and comparison
Explain my music	Describe the requested plan and uncertain detected features	Distinguish plan claims from observed audio
Adaptive music	Low/medium/high-energy versions that can transition	Tempo/key aligned stems and transition testing

These options should appear only when their prerequisites exist. A future feature can be demonstrated in a clearly labelled experiment without becoming a false promise in the production UI.

17. Delivery stages and decisions

Stage A, personal MVP: brief-to-plan, local ACE generation, useful instrument/voice/lyric choices, player, favourites, versions, exports and honest capability labels. Preserve Yamuna and Daylight as accepted project anchors. No public hosting required.

Stage B, controlled refinement: section repair pilot, richer section planning, instrument examples, language evaluation set and estimated stem workflows. Promote a feature only after it preserves what users value.

Stage C, advanced studio: optional paid adapters, score/humming input, independently controllable parts, video cues and broader language coverage. Test adapter-specific behavior; do not copy a website feature list into an API integration claim.

Open decisions for implementation: final name; planner provider and API budget; initial allowed generation length and number of takes; web-only versus packaged desktop; whether to include lyrics generation in the first build; which cloud integration, if any; privacy and retention defaults for a hosted version. Recommended default is to include lyrics drafting, keep local music inference and avoid requiring paid generation for the first usable release.

18. Sources and research scope

Reviewed current public documentation, not logged-in product trials. Vendor feature claims describe interfaces and do not establish comparative sound quality. No new music was generated for the PRD.

- **S1:** Suno Studio 2.0: https://help.suno.com/en/articles/13670529 . Inspiration: chat, alternate takes, real automation, MIDI and effects. Product subscription/features do not imply access through an official public API.
- **S2:** Suno Song Editor: https://help.suno.com/en/articles/6141505 . Inspiration: bounded lyric/section revisions and editable transitions.
- **S3:** SOUNDRAW: https://soundraw.io/ . Inspiration: approachable instrument, intensity and section editing; separate API/commercial terms apply.
- **S4:** Stable Audio official guide: https://kb.stability.ai/knowledge-base/how-to-use-stable-audio . Inspiration: explicit full-mix versus experimental agent multitrack modes, real mixing, and regenerate-one-part operations. Website features are not assumed to exist in every API.
- **S5:** Eleven composition plans: https://elevenlabs.io/docs/eleven-api/guides/how-to/music/composition-plans . Inspiration: section plans and local style directions.
- **S6:** AIVA: https://www.aiva.ai/ . Inspiration: audio/MIDI influences and editable compositions; not evidence for Hindi singing quality.
- **S7:** Udio Sessions: https://help.udio.com/en/articles/11649525-sessions-udio-s-timeline-editing-view ; export restriction: https://help.udio.com/en/articles/12683565-changes-associated-with-the-universal-music-group-umg-partnership . Timeline/take UX reference; current official notice disables downloads, so not an assumed export backend.
- **S8:** ACE inference fields: https://github.com/ace-step/ACE-Step-1.5/blob/main/acestep/inference.py . Current installed behavior remains authoritative for the MVP adapter.
- **S9:** ACE inference guide: https://github.com/ace-step/ACE-Step-1.5/blob/main/docs/en/INFERENCE.md . Checkpoint-specific capabilities need validation.
- **S10:** YuE2 music skill: https://github.com/multimodal-art-projection/YuE/blob/main/skills/yue2-music/SKILL.md . Score-control candidate; local whole-performance regeneration is different from waveform preservation.
- **S11:** Eleven inpainting: https://elevenlabs.io/docs/eleven-api/guides/how-to/music/inpainting ; official skill https://github.com/elevenlabs/skills/blob/main/music/SKILL.md . Existing-file inpainting upload entitlement must be checked; not interchangeable with ordinary style reference.
- **S12:** Suno terminology glossary: https://help.suno.com/en/articles/9010177 . Plain-language education reference; no guarantee every described technique is obeyed.
- **J1:** Ashes on the Fire: https://www.youtube.com/watch?v=v1Zz-dCbnEs . Analysis based on the transcript provided by the user.
- **J2:** YouSeeBIGGIRL/T:T: https://www.youtube.com/watch?v=IQRl_Li3Rz0 . Full English caption text retrieved through Agent Reach for internal review; not redistributed here.
- **J3:** La Signora Battle Theme: https://www.youtube.com/watch?v=hqHiD0a34F4 . Full English captions retrieved through Agent Reach; not redistributed here.
- **J4:** Requested channel: https://www.youtube.com/@joeynato_music . Agent Reach results identify the reviewed videos as Joey Nato, channel ID UC5hZbtg-Xu6gtPCWcEw3V3w.
- **R1:** Agent Reach: https://github.com/Panniantong/Agent-Reach . GitHub source pinned at beabda7486e9a666360b18cdb5c0e759c2c4d20f; installed into a separate project environment. Public search/subtitle operations only; no login, account changes, browser-session copying, paid transcription or global skill registration.

Companion files provide a prioritized control catalogue and example structured plan. They specify proposed behavior; they are not a functioning generation application.